EXPERIMENT
03

1 Aim

To design an **Employee–Department–Project** relational schema in MySQL, populate it with 30 employees across 5 departments and 8 projects, and write SQL queries demonstrating **selection, projection, aggregate functions, GROUP BY, HAVING, CASE expressions and ORDER BY**.

2 Software / Tools Required

Component	Specification
DBMS	MySQL Server 8.0 (Community Edition)
Query Editor	Online MySQL compiler / MySQL Workbench 8.0
Operating System	Windows 11
Language	SQL (DDL, DML, DQL)

3 Theory

Relational algebra in SQL. Every `SELECT` statement is a composition of relational-algebra operators. The `WHERE` clause implements **selection (σ)** — it keeps the rows (tuples) that satisfy a predicate. The column list after `SELECT` implements **projection (π)** — it keeps only the chosen attributes.

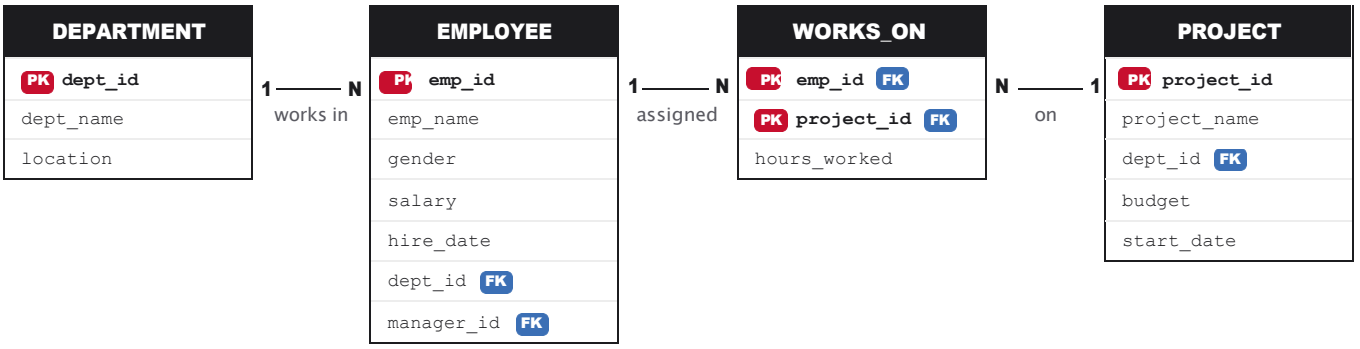
Aggregate functions — `COUNT, SUM, AVG, MAX, MIN` — reduce a set of rows to one value. `COUNT(*)` counts rows; the others ignore `NULL` s.

GROUP BY partitions rows into groups that share the same value of the grouping column(s), and the aggregate is computed once per group. **HAVING** filters those groups *aker* aggregation, while **WHERE** filters individual rows *before* grouping, so an aggregate condition such as `COUNT(*) >= 5` must go in HAVING.

CASE is SQL's conditional expression (like if-else). It evaluates conditions top-down and returns the first matching result, which makes it useful for derived columns such as salary bands. **ORDER BY** sorts the final result (`ASC` by default, `DESC` for descending).

4 Database / Table Structure

The schema has four relations. `works on` resolves the many-to-many relationship between employees and projects, and `manager id` is a self-referencing foreign key that models the reporting hierarchy.



PK Primary key · FK Foreign key · `employee.manager_id → employee.emp_id` (recursive 1:N) · `project.dept id → department.dept id`

Table: department

Column	Data Type	Constraint	Description
`dept_id`	INT	PRIMARY KEY	Unique department number
`dept_name`	VARCHAR(50)	NOT NULL, UNIQUE	Department name
`location`	VARCHAR(50)	NOT NULL	Office city

Table: employee

Column	Data Type	Constraint	Description
`emp_id`	INT	PRIMARY KEY	Unique employee number
`emp_name`	VARCHAR(50)	NOT NULL	Employee name
`gender`	ENUM('Male','Female')	NOT NULL	Gender
`salary`	DECIMAL(10,2)	NOT NULL, CHECK > 0	Monthly salary (₹)
`hire_date`	DATE	NOT NULL	Date of joining
`dept_id`	INT	FK → department	Department of employee
`manager_id`	INT	FK → employee (self)	Reporting manager

Table: project

Column	Data Type	Constraint	Description
`project_id`	INT	PRIMARY KEY	Unique project number
`project_name`	VARCHAR(60)	NOT NULL, UNIQUE	Project title
`dept_id`	INT	FK → department	Owning department
`budget`	DECIMAL(12,2)	NOT NULL	Sanctioned budget (₹)
`start_date`	DATE	NOT NULL	Project start date

Table: works_on

Column	Data Type	Constraint	Description
`emp_id`	INT	PK, FK → employee	Assigned employee
`project_id`	INT	PK, FK → project	Assigned project
`hours_worked`	INT	NOT NULL, DEFAULT 0	Hours logged

Data loaded (30 employees · 5 departments · 8 projects · 30 assignments):

ID	Department	Location	Employees	Head	Projects owned
1	IT	Noida	6	Aarav	ERP Development, Website Redesign, Mobile Application
2	HR	Lucknow	6	Rohan	Payroll System
3	Finance	Mumbai	6	Arjun	Data Migration
4	Marketing	Delhi	6	Varun	Marketing Campaign, Customer Analytics
5	Operations	Bengaluru	6	Mohit	Cloud Infrastructure

5 SQL Commands

5.1 DDL — create tables

```sql
-- Create and select the database
CREATE DATABASE IF NOT EXISTS company_db;
USE company_db;

-- 1. DEPARTMENT
CREATE TABLE department (
    dept_id     INT           PRIMARY KEY,
    dept_name   VARCHAR(50)   NOT NULL UNIQUE,
    location    VARCHAR(50)   NOT NULL
);

-- 2. EMPLOYEE  (self-referencing FK for manager)
CREATE TABLE employee (
    emp_id      INT             PRIMARY KEY,
```

```
    emp_name     VARCHAR(50)     NOT NULL,
    gender       ENUM('Male','Female') NOT NULL,
    salary       DECIMAL(10,2)  NOT NULL CHECK (salary > 0),
    hire_date    DATE           NOT NULL,
    dept_id      INT,
    manager id   INT,
    CONSTRAINT fk_emp_dept FOREIGN KEY (dept_id)
        REFERENCES department(dept_id) ON DELETE SET NULL,
    CONSTRAINT fk_emp_mgr  FOREIGN KEY (manager_id)
        REFERENCES employee(emp_id)     ON DELETE SET NULL
);

-- 3. PROJECT
CREATE TABLE project (
    project_id     INT            PRIMARY KEY,
    project_name   VARCHAR(60)    NOT NULL UNIQUE,
    dept_id        INT            NOT NULL,
    budget         DECIMAL(12,2)  NOT NULL,
    start_date     DATE           NOT NULL,
    CONSTRAINT fk_proj_dept FOREIGN KEY (dept_id)
        REFERENCES department(dept_id)
);

-- 4. WORKS_ON  (M:N bridge between EMPLOYEE and PROJECT)
CREATE TABLE works_on (
    emp_id         INT,
    project_id     INT,
    hours_worked  INT NOT NULL DEFAULT 0,
    PRIMARY KEY (emp_id, project_id),
    CONSTRAINT fk_wo_emp  FOREIGN KEY (emp_id)
        REFERENCES employee(emp_id)    ON DELETE CASCADE,
    CONSTRAINT fk_wo_proj FOREIGN KEY (project_id)
        REFERENCES project(project_id) ON DELETE CASCADE
);
```

5.2 DML — insert sample data

```
-- 5 departments
INSERT INTO department (dept_id, dept_name, location) VALUES
    (1, 'IT', 'Noida'),
    (2, 'HR', 'Lucknow'),
    (3, 'Finance', 'Mumbai'),
    (4, 'Marketing', 'Delhi'),
    (5, 'Operations', 'Bengaluru');

-- 30 employees (6 per department)
INSERT INTO employee
    (emp_id, emp_name, gender, salary, hire_date, dept_id, manager_id)
VALUES
    (101, 'Aarav', 'Male', 65000, '2021-01-15', 1, NULL),
    (102, 'Priya', 'Female', 58000, '2021-03-20', 1, 101),
    (103, 'Rahul', 'Male', 72000, '2020-07-10', 1, 101),
    (104, 'Sneha', 'Female', 55000, '2022-02-11', 1, 103),
    (105, 'Aditya', 'Male', 48000, '2023-06-05', 1, 103),
    (106, 'Neha', 'Female', 62000, '2020-05-18', 1, 101),
    (107, 'Rohan', 'Male', 52000, '2019-08-01', 2, NULL),
    (108, 'Ananya', 'Female', 60000, '2020-09-14', 2, 107),
    (109, 'Karan', 'Male', 45000, '2023-01-09', 2, 107),
    (110, 'Ishita', 'Female', 57000, '2021-11-22', 2, 108),
```

```
    (111, 'Vivek', 'Male', 49000, '2022-07-18', 2, 108),
    (112, 'Meera', 'Female', 68000, '2019-11-25', 2, 107),
    (113, 'Arjun', 'Male', 75000, '2020-02-14', 3, NULL),
    (114, 'Kavya', 'Female', 65000, '2021-05-17', 3, 113),
    (115, 'Manish', 'Male', 55000, '2022-03-28', 3, 113),
    (116, 'Pooja', 'Female', 59000, '2021-09-06', 3, 114),
    (117, 'Sahil', 'Male', 51000, '2023-04-12', 3, 114),
    (118, 'Divya', 'Female', 70000, '2020-08-19', 3, 113),
    (119, 'Varun', 'Male', 58000, '2019-12-02', 4, NULL),
    (120, 'Riya', 'Female', 62000, '2020-06-16', 4, 119),
    (121, 'Aditi', 'Female', 56000, '2022-05-23', 4, 119),
    (122, 'Kunal', 'Male', 48000, '2023-02-15', 4, 120),
    (123, 'Simran', 'Female', 46000, '2023-08-21', 4, 120),
    (124, 'Tanya', 'Female', 66000, '2020-12-01', 4, 119),
    (125, 'Mohit', 'Male', 61000, '2021-02-10', 5, NULL),
    (126, 'Yash', 'Male', 54000, '2022-01-17', 5, 125),
    (127, 'Naina', 'Female', 52000, '2022-10-03', 5, 125),
    (128, 'Rajat', 'Male', 50000, '2023-03-14', 5, 126),
    (129, 'Nikhil', 'Male', 51000, '2023-07-24', 5, 126),
    (130, 'Sakshi', 'Female', 67000, '2020-10-09', 5, 125);

-- 8 projects
INSERT INTO project (project_id, project_name, dept_id, budget, start_date) VALUES
    (1, 'ERP Development', 1, 1500000, '2024-01-10'),
    (2, 'Website Redesign', 1, 600000, '2024-02-01'),
    (3, 'Mobile Application', 1, 900000, '2024-03-15'),
    (4, 'Data Migration', 3, 750000, '2024-04-01'),
    (5, 'Marketing Campaign', 4, 500000, '2024-05-20'),
    (6, 'Payroll System', 2, 650000, '2024-06-10'),
    (7, 'Customer Analytics', 4, 800000, '2024-07-05'),
    (8, 'Cloud Infrastructure', 5, 1200000, '2024-08-12');

-- Employee-project assignments
INSERT INTO works_on (emp_id, project_id, hours_worked) VALUES
    (101, 1, 120),
    (102, 1, 95),
    (103, 1, 110),
    (126, 1, 80),
    (104, 2, 60),
    (105, 2, 140),
    (127, 2, 75),
    (106, 3, 90),
    (117, 3, 100),
    (128, 3, 85),
    (107, 4, 120),
    (108, 4, 95),
    (118, 4, 110),
    (130, 4, 80),
    (109, 5, 60),
    (110, 5, 140),
    (119, 5, 75),
    (120, 5, 90),
    (122, 5, 100),
    (111, 6, 85),
    (112, 6, 120),
    (113, 6, 95),
    (115, 6, 110),
    (114, 7, 80),
    (116, 7, 60),
    (121, 7, 140),
```

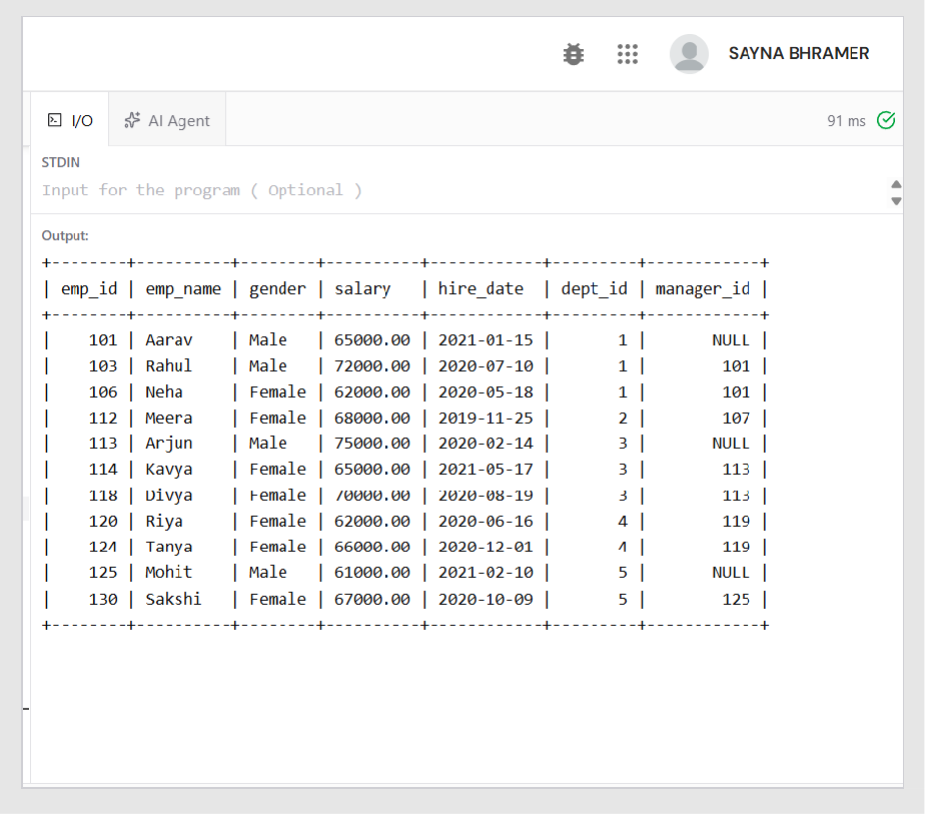
```
    (123, 7, 75),
    (129, 7, 90),
    (124, 8, 100),
    (125, 8, 85);
```

5.3 Queries — each is shown with its output in the next section.

6 Output

Q3.1 Selection (σ) — employees earning more than ₹60,000

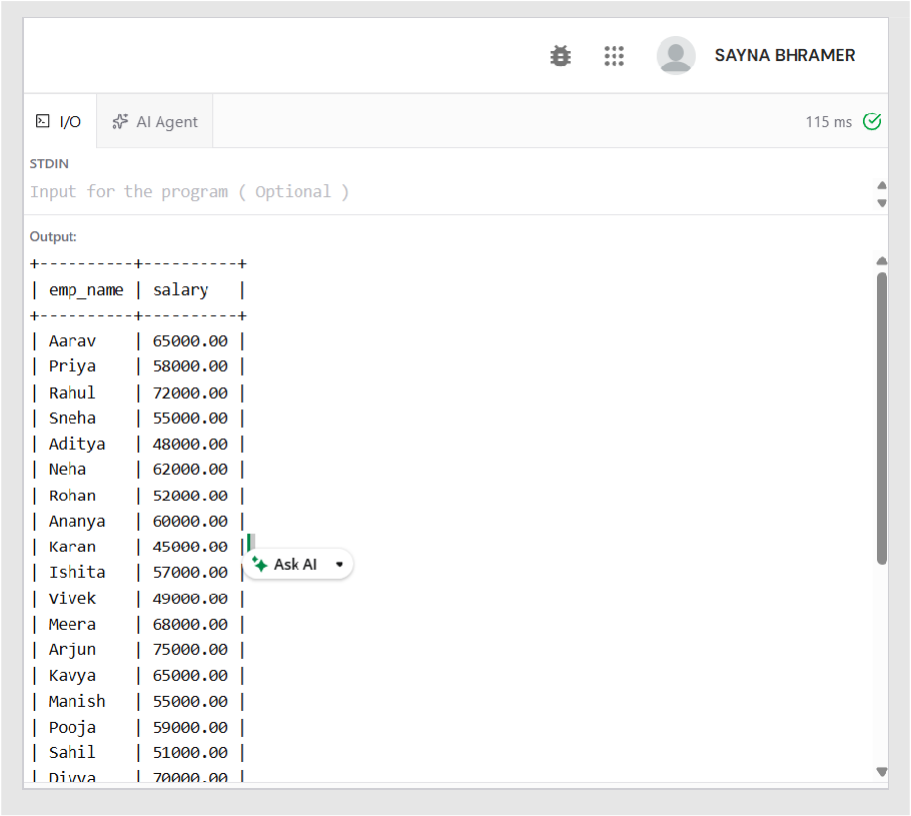
```
SELECT *
FROM employee
WHERE salary > 60000;
```




```
+--------+----------+--------+----------+------------+---------+------------+
| emp_id | emp_name | gender | salary   | hire_date  | dept_id | manager_id |
+--------+----------+--------+----------+------------+---------+------------+
|    101 | Aarav    | Male   | 65000.00 | 2021-01-15 |       1 |       NULL |
|    103 | Rahul    | Male   | 72000.00 | 2020-07-10 |       1 |        101 |
|    106 | Neha     | Female | 62000.00 | 2020-05-18 |       1 |        101 |
|    112 | Meera    | Female | 68000.00 | 2019-11-25 |       2 |        107 |
|    113 | Arjun    | Male   | 75000.00 | 2020-02-14 |       3 |       NULL |
|    114 | Kavya    | Female | 65000.00 | 2021-05-17 |       3 |        113 |
|    118 | Divya    | Female | 70000.00 | 2020-08-19 |       3 |        113 |
|    120 | Riya     | Female | 62000.00 | 2020-06-16 |       4 |        119 |
|    124 | Tanya    | Female | 66000.00 | 2020-12-01 |       4 |        119 |
|    125 | Mohit    | Male   | 61000.00 | 2021-02-10 |       5 |       NULL |
|    130 | Sakshi   | Female | 67000.00 | 2020-10-09 |       5 |        125 |
+--------+----------+--------+----------+------------+---------+------------+
```

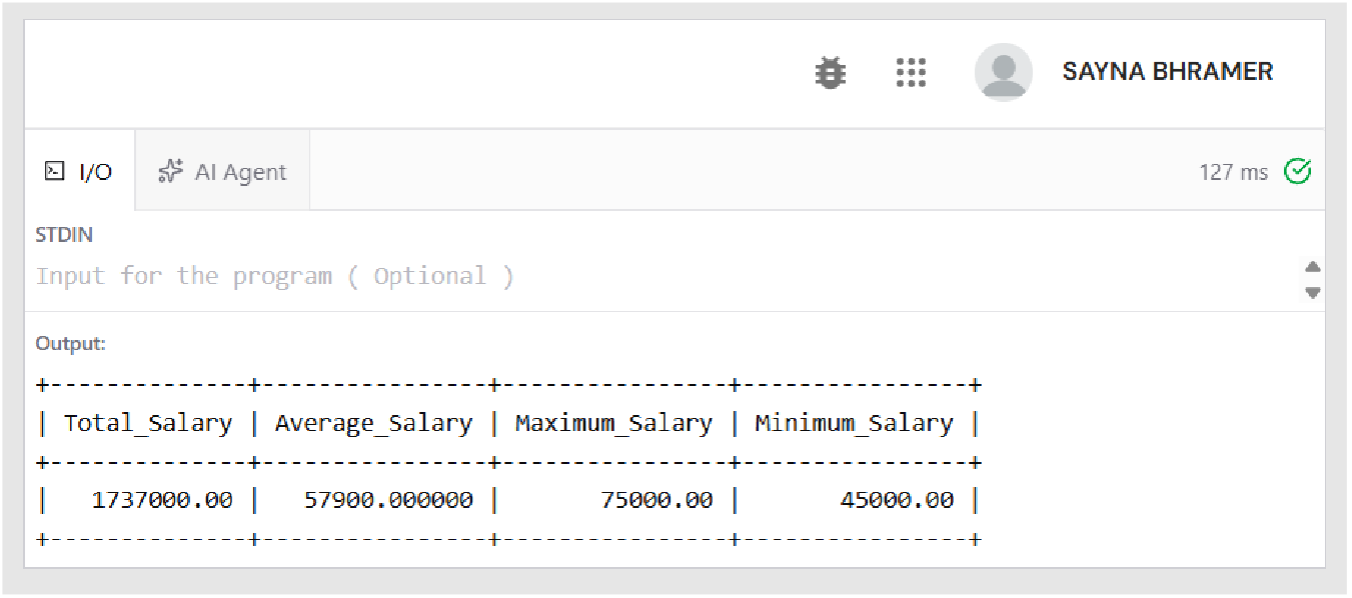
Q3.2 Projection (π) — only name and salary columns

```
SELECT emp_name, salary
FROM employee;
```



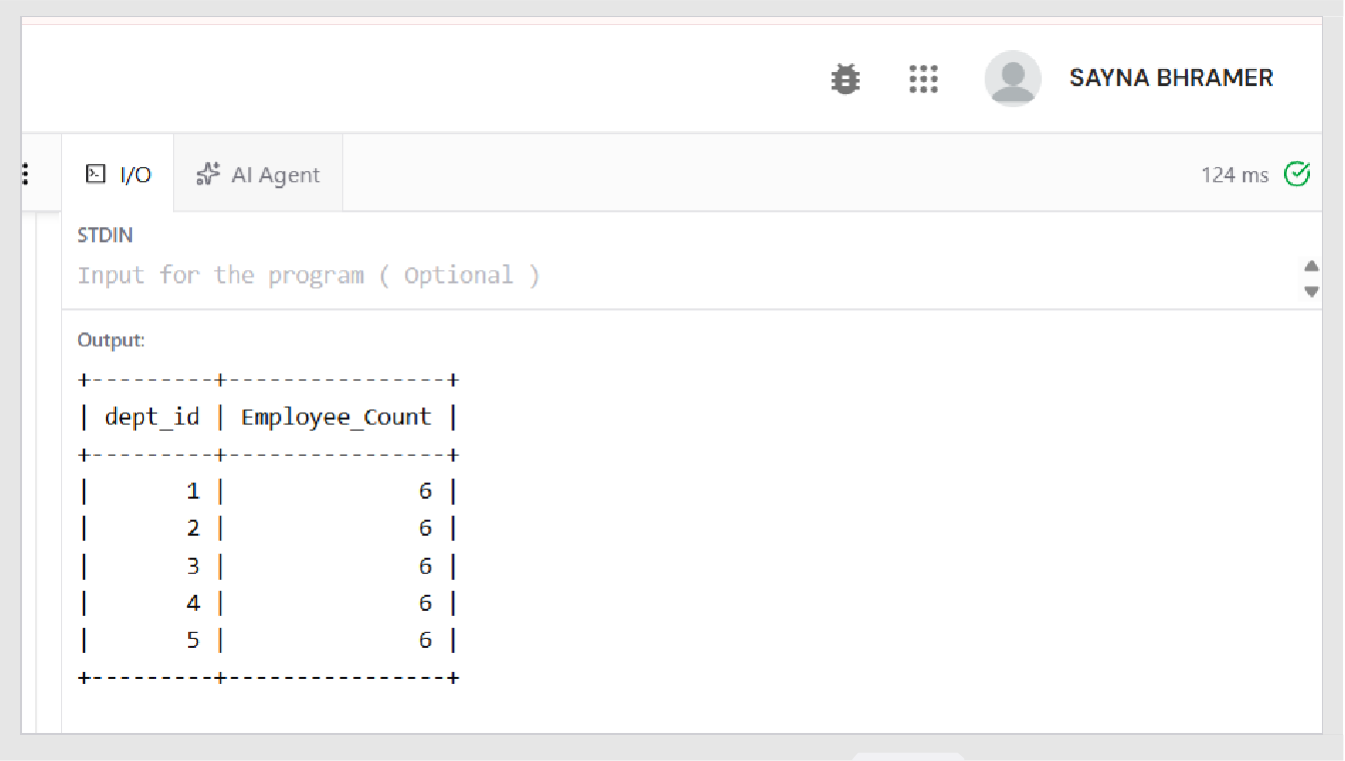
Q3.3 Aggregate functions — company-wide salary statistics

```sql
SELECT SUM(salary) AS Total_Salary,
       AVG(salary) AS Average_Salary,
       MAX(salary) AS Maximum_Salary,
       MIN(salary) AS Minimum_Salary
FROM employee;
```



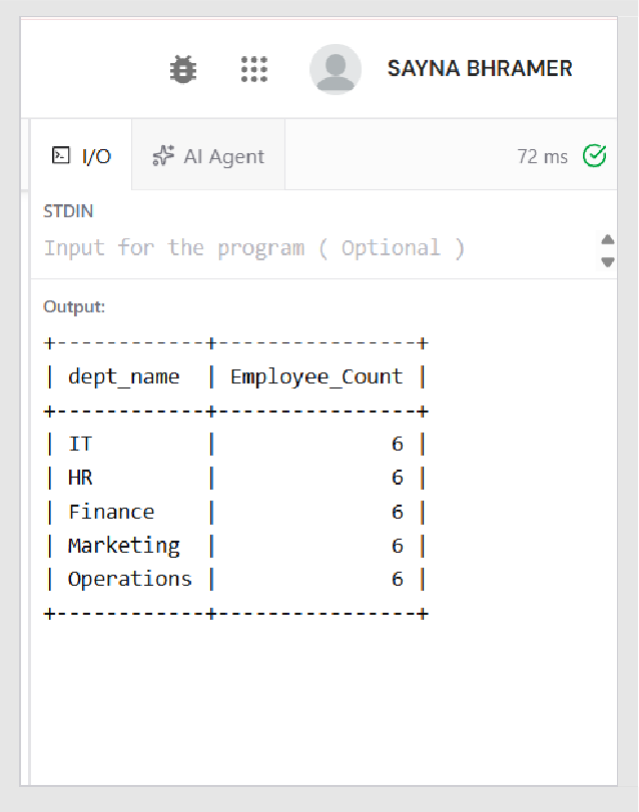
Q3.4 GROUP BY — head-count per department

```sql
SELECT dept_id, COUNT(*) AS Employee_Count
FROM employee
GROUP BY dept_id;
```



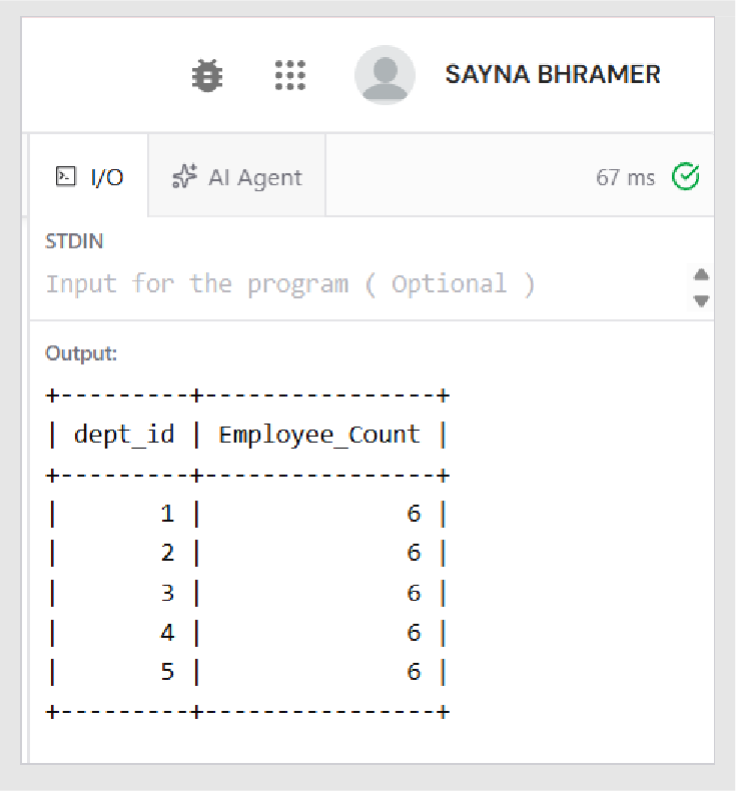
Q3.5 GROUP BY with department names

```sql
SELECT d.dept_name, COUNT(e.emp_id) AS Employee_Count
FROM department d
JOIN employee e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```



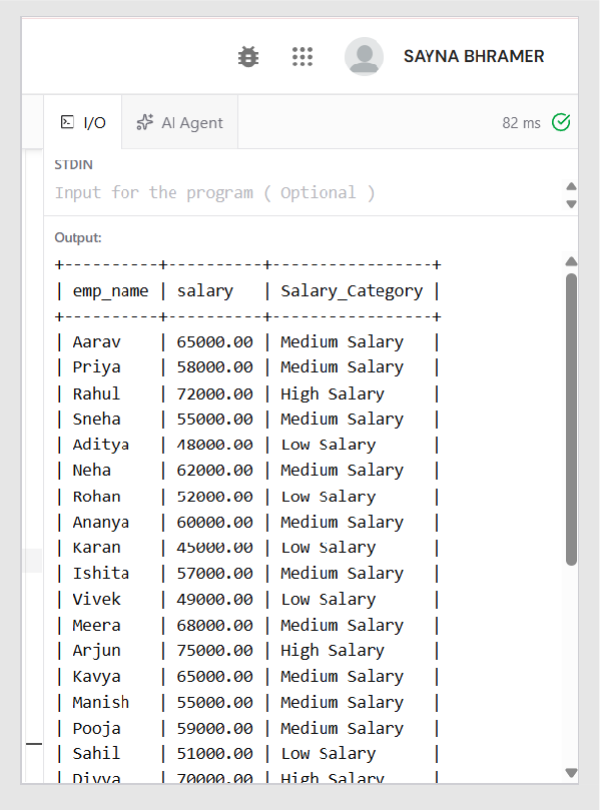
Q3.6 HAVING — departments with at least 5 employees

```sql
SELECT dept_id, COUNT(*) AS Employee_Count
FROM employee
GROUP BY dept_id
HAVING COUNT(*) >= 5;
```



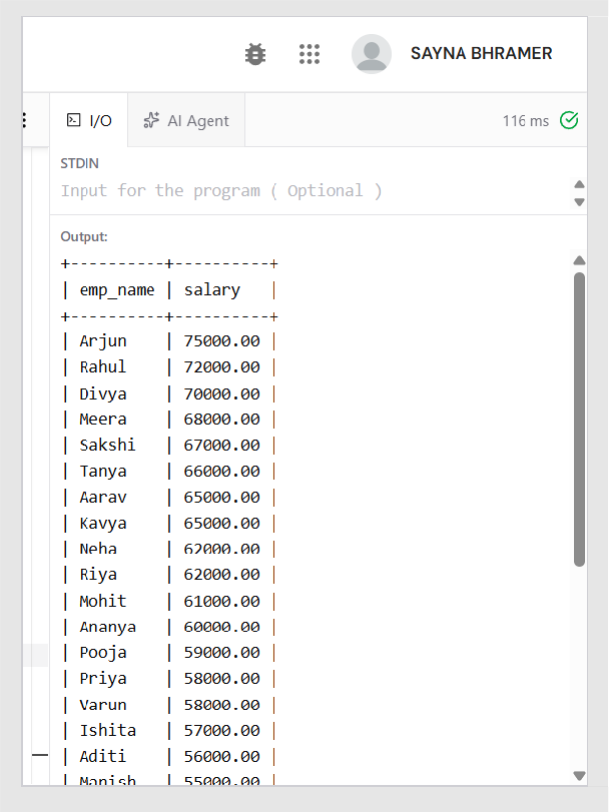
Q3.7 CASE expression — salary banding

```sql
SELECT emp_name, salary,
       CASE
           WHEN salary >= 70000 THEN 'High Salary'
           WHEN salary >= 55000 THEN 'Medium Salary'
           ELSE 'Low Salary'
       END AS Salary_Category
FROM employee;
```



Q3.8 ORDER BY — employees ranked by salary (highest first)

```sql
SELECT emp_name, salary
FROM employee
ORDER BY salary DESC;
```



7 Result / Conclusion

The Employee–Department–Project schema was created with primary-key, foreign-key, NOT NULL, UNIQUE and CHECK constraints, and populated with 30 employees, 5 departments and 8 projects. All eight queries ran and returned the expected results:

- 11 employees earn above ₹60,000. Total payroll is ₹17,37,000, with an average of ₹57,900, a maximum of ₹75,000 (Arjun) and a minimum of ₹45,000 (Karan).
- Each of the 5 departments has exactly 6 employees, so all of them pass the `HAVING COUNT(*) >= 5` filter.
- The CASE expression put 3 employees in the High band (≥ ₹70,000), 15 in Medium (≥ ₹55,000) and 12 in Low.

EXPERIMENT
04

1 Aim

Using the Employee schema from Experiment 3, to write queries using **INNER JOIN, LEFT JOIN, self-join, a 3-way join, correlated subqueries, EXISTS**, and **simulated INTERSECT and EXCEPT**, and to compare query execution plans using **EXPLAIN**.

2 Software / Tools Required

Component	Specification
DBMS	MySQL Server 8.0 (Community Edition)
Query Editor	Online MySQL compiler / MySQL Workbench 8.0
Operating System	Windows 11
Language	SQL (DDL, DML, DQL)
Plan analysis	EXPLAIN / EXPLAIN FORMAT=TREE

3 Theory

Joins combine rows from two or more tables on a related column.

Join type	Returns	Used here for
INNER JOIN	Only rows that match in both tables	Employee with department name
LEFT (OUTER) JOIN	All rows of the left table; NULLs where there is no match	All departments, even empty ones
SELF JOIN	A table joined with an alias of itself	Employee → manager
Multi-way JOIN	A chain of joins across 3+ tables	Employee × Department × Project

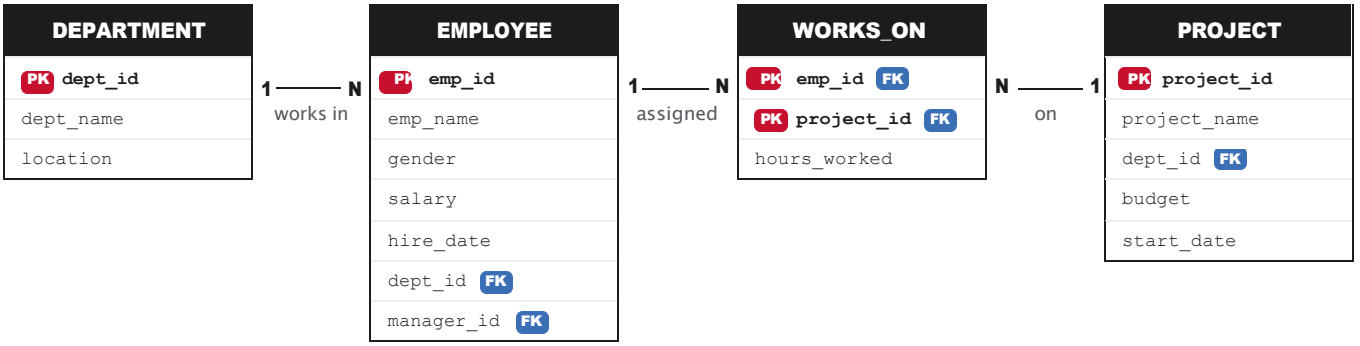
Subqueries. A *non-correlated* subquery runs once. A **correlated subquery** refers to a column of the outer query, so conceptually it is re-evaluated for every outer row. **EXISTS** returns TRUE as soon as the subquery produces one row, which makes it efficient for existence tests and safe with NULLs (unlike `NOT IN`).

Set operations. `INTERSECT` (A ∩ B) and `EXCEPT` (A − B) are only available from MySQL 8.0.31 onwards. They are portably simulated with `IN` / `EXISTS` (for intersection) and `NOT IN` / `NOT EXISTS` / `LEFT JOIN … IS NULL` (for difference).

EXPLAIN shows the optimizer's execution plan without running the query. Key columns: `select type` (SIMPLE, PRIMARY, SUBQUERY, **DEPENDENT SUBQUERY**, DERIVED), `type` (access method: ALL = full scan, ref, eq_ref, const — best), `possible_keys` / `key` (indexes considered/used), `rows` (estimated rows examined) and `Extra` (Using where, Using temporary, Using filesort).

4 Database / Table Structure

The same four relations from Experiment 3 are used; no new tables are created.



PK Primary key FK Foreign key · `employee.manager_id → employee.emp_id` (recursive 1:N) · `project.dept id → department.dept id`

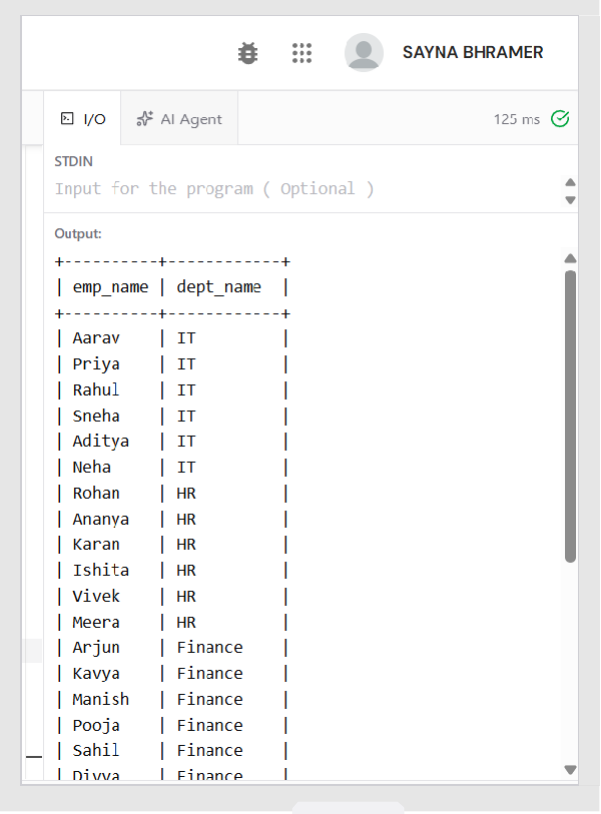
5 SQL Commands

The queries are listed below together with their outputs. Tables and data are the same as in Experiment 3.

6 Output

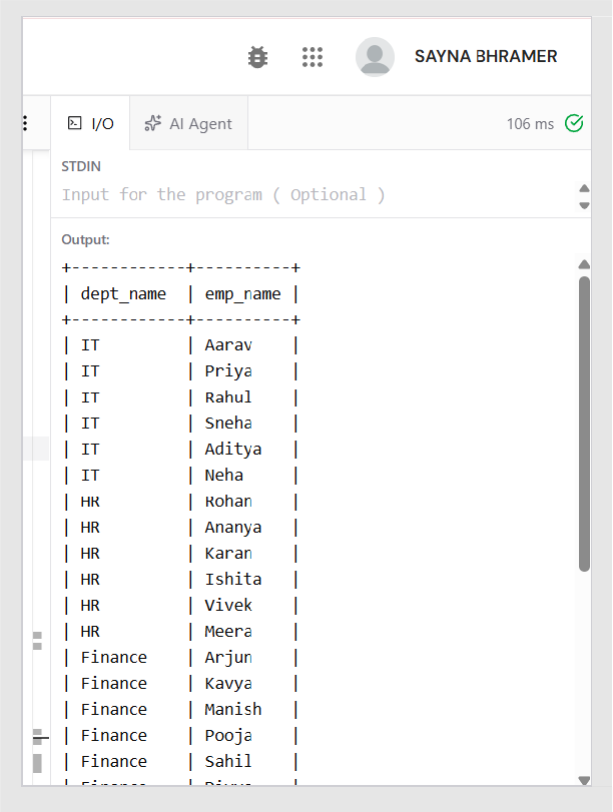
Q4.1 INNER JOIN — employee with department name

```
SELECT e.emp_name, d.dept_name
FROM employee e
INNER JOIN department d
        ON e.dept_id = d.dept_id;
```



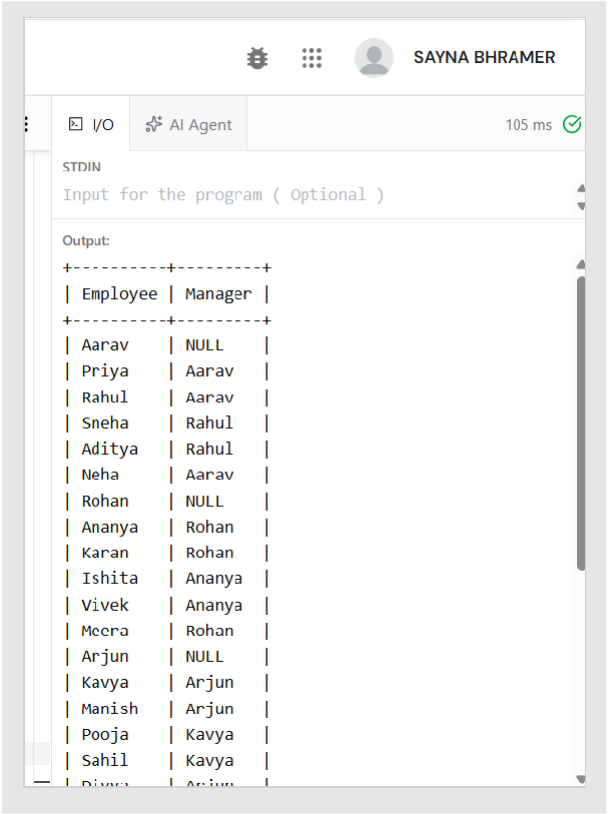
Q4.2 LEFT JOIN — every department with its employees

```sql
SELECT d.dept_name, e.emp_name
FROM department d
LEFT JOIN employee e
       ON d.dept_id = e.dept_id;
```



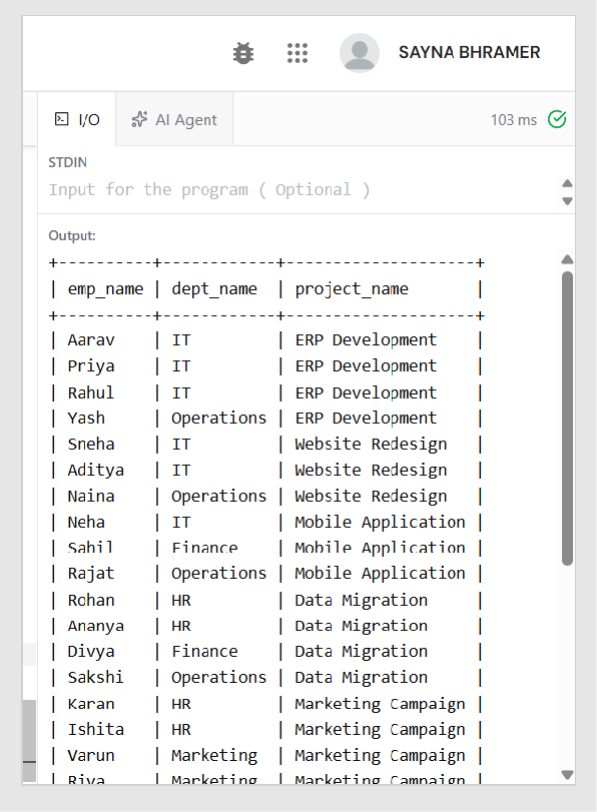
Q4.3 SELF JOIN — employee and reporting manager

```sql
SELECT e.emp_name AS Employee,
       m.emp_name AS Manager
FROM employee e
LEFT JOIN employee m
       ON e.manager_id = m.emp_id;
```



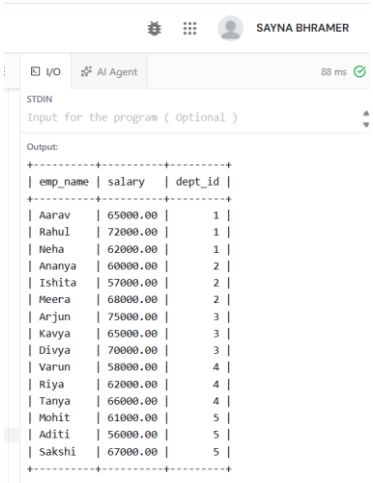
Q4.4 3-way JOIN — Employee × Department × Project

```
SELECT e.emp_name, d.dept_name, p.project_name
FROM works_on w
INNER JOIN employee    e ON w.emp id      = e.emp_id
INNER JOIN department d ON e.dept_id      = d.dept id
INNER JOIN project     p ON w.project_id = p.project_id
ORDER BY p.project id, e.emp id;
```



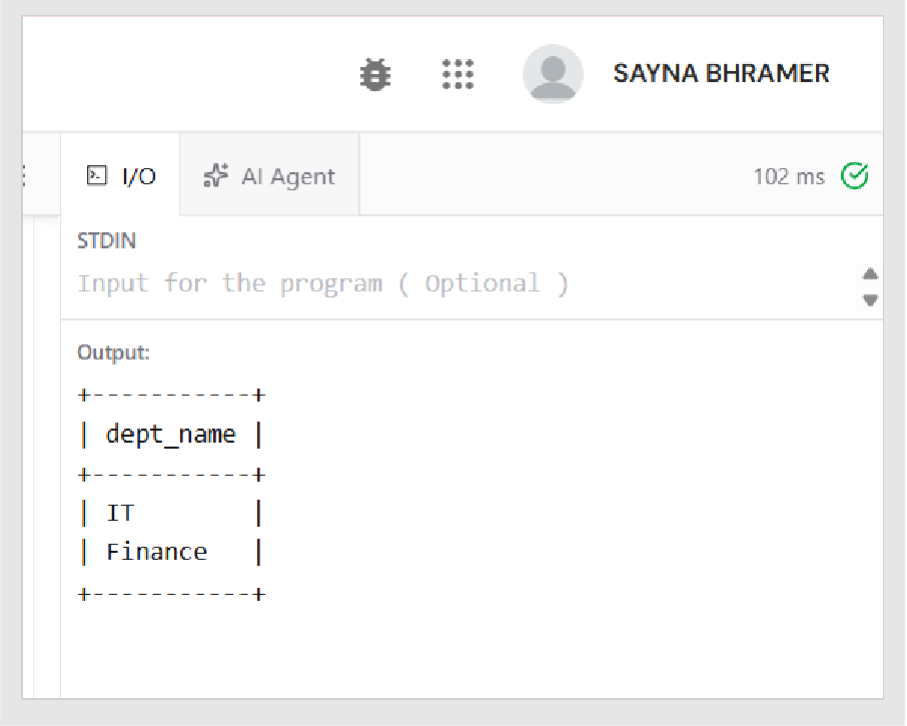
Q4.5 Correlated subquery — employees earning above their department average

```
SELECT e.emp_name, e.dept_id, e.salary
FROM employee e
WHERE e.salary > (SELECT AVG(i.salary)
                  FROM employee i
                  WHERE i.dept_id = e.dept_id);
```



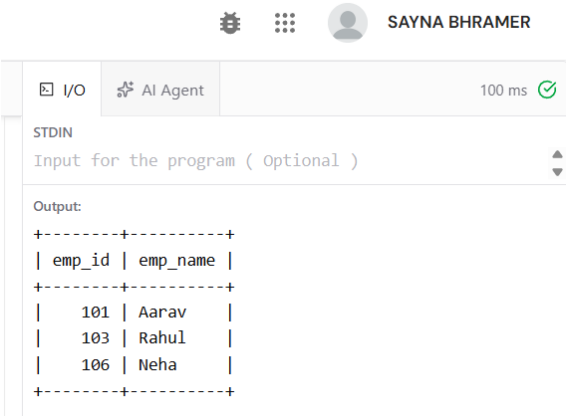
Q4.6 EXISTS — departments having at least one employee earning above ₹70,000

```sql
SELECT d.dept_name
FROM department d
WHERE EXISTS (SELECT 1
              FROM employee e
              WHERE e.dept_id = d.dept_id
                AND e.salary > 70000);
```



Q4.7 Simulated INTERSECT — IT employees who also work on ERP Development

```sql
SELECT e.emp_id, e.emp_name
FROM employee e
WHERE e.dept_id = 1
   AND e.emp_id IN (SELECT w.emp_id
                    FROM works_on w
                    JOIN project p ON w.project_id = p.project_id
                    WHERE p.project_name = 'ERP Development');
```



Q4.8 Simulated EXCEPT — high earners (> ₹60,000) who are NOT in IT

```sql
SELECT emp_id, emp_name
FROM employee
WHERE salary > 60000
   AND emp_id NOT IN (SELECT emp_id
                      FROM employee
                      WHERE dept_id = 1);
```



Q4.9 EXPLAIN — plan of the correlated subquery (Q4.5)

```sql
EXPLAIN
SELECT e.emp_name, e.dept_id, e.salary
FROM employee e
WHERE e.salary > (SELECT AVG(i.salary)
                  FROM employee i
                  WHERE i.dept_id = e.dept_id);
```



Q4.10 EXPLAIN — same result rewritten as a JOIN with a derived table

```
EXPLAIN
SELECT e.emp_name, e.dept_id, e.salary
FROM employee e
JOIN (SELECT dept_id, AVG(salary) AS avg_sal
      FROM employee
      GROUP BY dept_id) a ON e.dept_id = a.dept_id
WHERE e.salary > a.avg_sal;
```



Execution plan comparison

Aspect	Correlated subquery (Q4.9)	JOIN + derived table (Q4.10)
select_type	PRIMARY + DEPENDENT SUBQUERY	PRIMARY + DERIVED
Inner work	Re-evaluated for each outer row (30×)	Averages computed once (5 groups)
Scalability	Cost grows ≈ $O(n^2)$ on large tables	Cost grows ≈ $O(n)$ + join
Verdict	Readable, fine for small data	Preferred for large data

7 Result / Conclusion

All join types, subqueries and simulated set operations ran and returned the expected results:

- The self-join exposed the three-level reporting hierarchy.
- The 3-way join showed that projects span departments; for example, Yash from Operations works on ERP Development.
- EXISTS returned only IT and Finance, the two departments with an employee earning above ₹70,000.
- Simulated EXCEPT returned 8 employees: the 11 high earners minus the 3 in IT.

EXPLAIN showed that the correlated subquery runs as a *DEPENDENT SUBQUERY*, re-executed for every outer row. Rewriting it as a join with a derived table computes each department average only once, which is the better choice for large tables.

EXPERIMENT
05

1 Aim

To create SQL **views** for a department salary summary and the employee hierarchy, to **test the updatability** of views, and to implement a **recursive Common Table Expression (CTE)** that displays the complete reporting chain of every employee.

2 Software / Tools Required

Component	Specification
DBMS	MySQL Server 8.0 (Community Edition)
Query Editor	Online MySQL compiler / MySQL Workbench 8.0
Operating System	Windows 11
Language	SQL (DDL, DML, DQL)
Feature needed	WITH RECURSIVE (MySQL ≥ 8.0)

3 Theory

A **view** is a named, stored `SELECT` query that behaves like a virtual table: it stores no data of its own and is re-evaluated each time it is queried. Views provide **abstraction** (they hide complex joins), **security** (they expose only selected rows and columns) and **logical data independence**.

Updatable views. MySQL allows INSERT, UPDATE and DELETE through a view only if each view row maps to exactly one base-table row. A view is **not updatable** if it contains:

Makes a view read-only	Reason
Aggregate functions (SUM, AVG, COUNT…)	A view row summarises many base rows
GROUP BY / HAVING / DISTINCT	No one-to-one row mapping
UNION / UNION ALL	Row origin is ambiguous
Subquery in SELECT list, derived tables	The value is computed, not stored
ALGORITHM = TEMPTABLE	The view is materialised in a temporary table

`WITH CHECK OPTION` rejects any change made through the view that would make the row fall outside the view's `WHERE` condition.

Recursive CTE. `WITH RECURSIVE` defines a CTE that refers to itself. It has an **anchor member** (the starting rows, here the managers with `manager_id IS NULL`) combined by `UNION ALL` with a **recursive member** that joins the CTE back to the base table. Each iteration adds the next level of the hierarchy, and recursion stops when an iteration returns no new rows.

4 Database / Table Structure

Base tables `employee` and `department` from Experiment 3. Objects created in this experiment:

Object	Type	Based on	Updatable?
`dept_salary_summary`	View	department ⋈ employee, GROUP BY	No (aggregates)
`employee_hierarchy`	View	employee (self-join) ⋈ department	Partially (key-preserved columns)
`it_employees`	View + CHECK OPTION	employee WHERE dept_id = 1	Yes
`reporting_chain`	Recursive CTE	employee.manager_id → emp_id	— (query-scoped)

5 SQL Commands

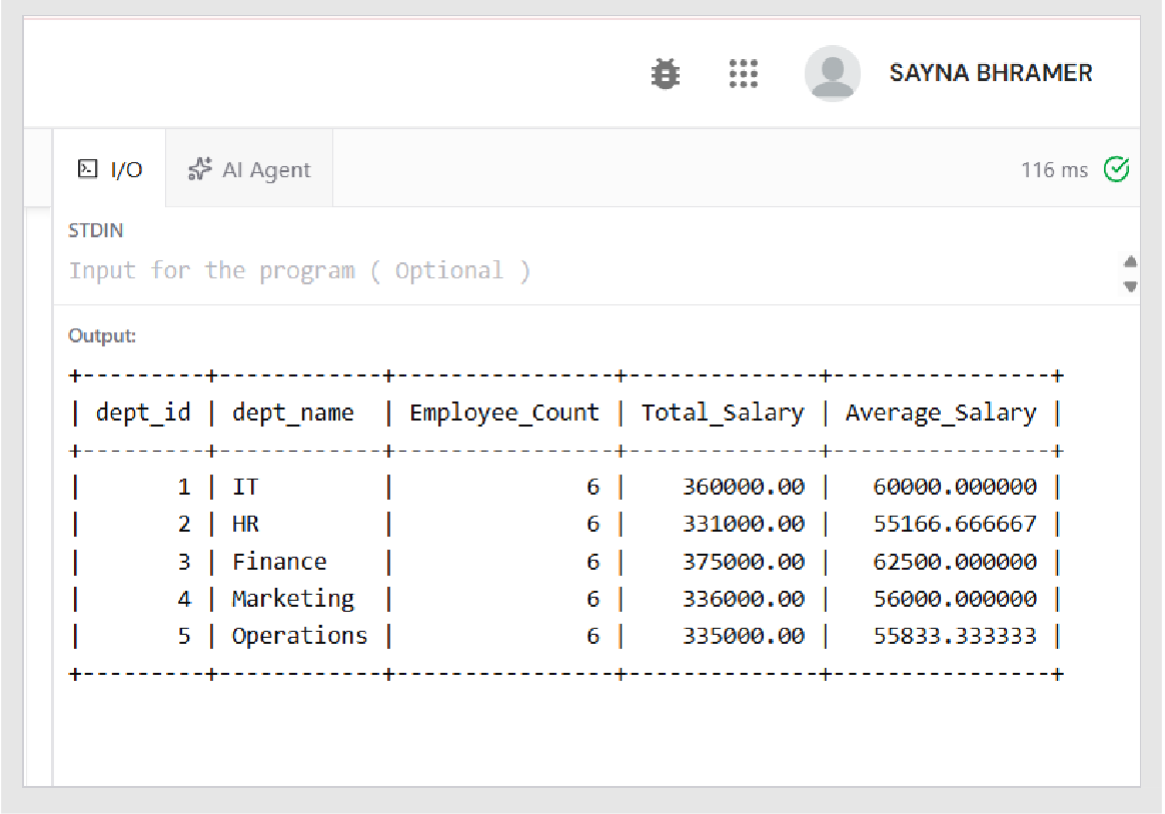
Each command block is shown with its output below.

6 Output

Q5.1 View 1 — department salary summary

```
CREATE VIEW dept_salary_summary AS
SELECT d.dept_id,
       d.dept_name,
       COUNT(e.emp_id) AS Employee_Count,
       SUM(e.salary)   AS Total_Salary,
       AVG(e.salary)   AS Average_Salary
FROM department d
LEFT JOIN employee e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;

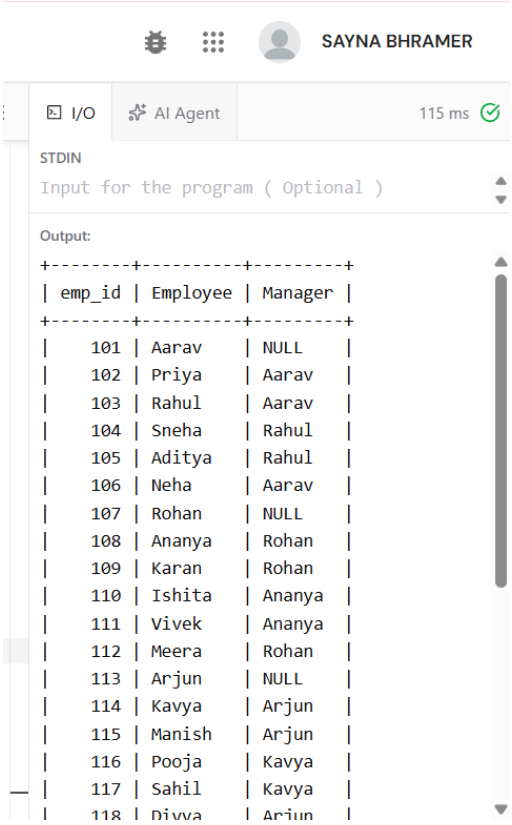
SELECT * FROM dept_salary_summary;
```



Q5.2 View 2 — employee hierarchy (employee → manager)

```sql
CREATE VIEW employee_hierarchy AS
SELECT e.emp_id,
       e.emp_name  AS Employee,
       m.emp_name  AS Manager,
       d.dept_name AS Department
FROM employee e
LEFT JOIN employee    m ON e.manager_id = m.emp_id
INNER JOIN department d ON e.dept_id    = d.dept_id;

SELECT * FROM employee_hierarchy;
```

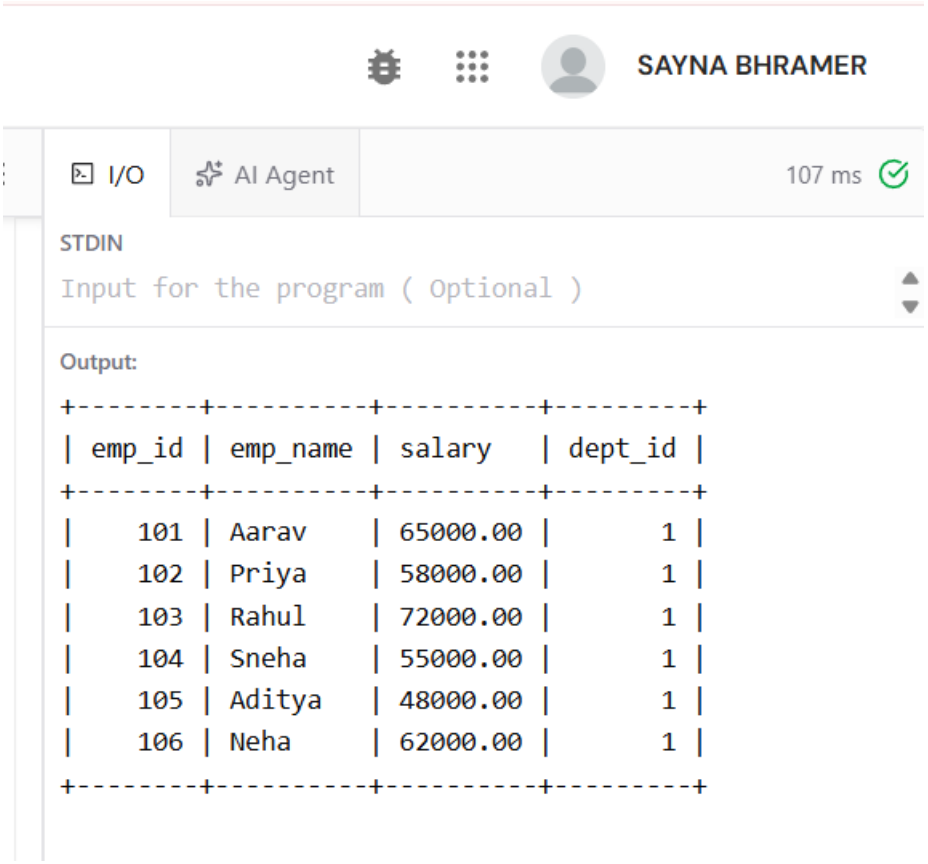


Q5.3 Updatable view — simple view on one table, WITH CHECK OPTION

```sql
CREATE VIEW it_employees AS
SELECT emp_id, emp_name, salary, dept_id
FROM employee
WHERE dept_id = 1
WITH CHECK OPTION;

-- Update through the view (reaches the base table)
UPDATE it_employees SET salary = salary + 2000 WHERE emp_id = 105;
SELECT * FROM it_employees;
SELECT emp_id, emp_name, salary FROM employee WHERE emp_id = 105;

-- Restore the original value
UPDATE it_employees SET salary = salary - 2000 WHERE emp_id = 105;
```

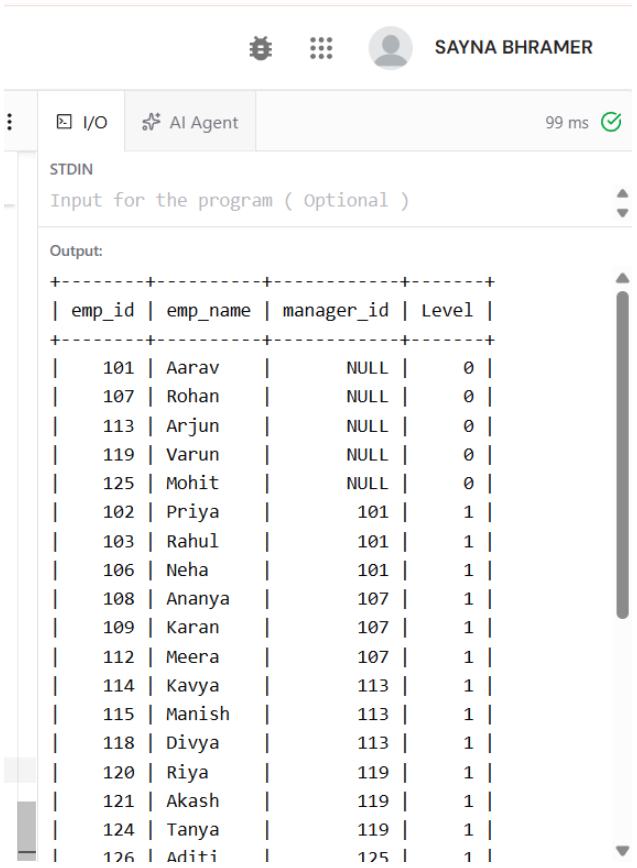


Q5.4 Recursive CTE — full reporting chain from each department head

```sql
WITH RECURSIVE reporting_chain AS (
    -- anchor: top-level managers
    SELECT emp_id, emp_name, manager_id,
           1 AS level,
           CAST(emp_name AS CHAR(200)) AS chain
    FROM employee
    WHERE manager_id IS NULL

    UNION ALL

    -- recursive step: direct reports of the previous level
    SELECT e.emp_id, e.emp_name, e.manager_id,
           rc.level + 1,
           CONCAT(rc.chain, ' -> ', e.emp_name)
    FROM employee e
    INNER JOIN reporting_chain rc ON e.manager_id = rc.emp_id
)
SELECT emp_id, emp_name, level, chain AS Reporting_Chain
FROM reporting_chain
ORDER BY chain;
```



7 Result / Conclusion

The two views `dept_salary_summary` and `employee_hierarchy` were created and queried like ordinary tables.

- The summary view showed Finance with the highest total payroll (₹3,75,000) and HR with the lowest average (₹55,166.67).
- Updatability tests confirmed that a simple single-table view (`it_employees`) passes updates through to the base table, and that `WITH CHECK OPTION` blocks changes that would move a row out of the view.
- The aggregate view is read-only (ERROR 1288).
- The recursive CTE listed the full reporting chain for all 30 employees across three levels, for example Arjun → Kavya → Pooja.

EXPERIMENT
06

1 Aim

To create a stored procedure `transfer_employee(emp_id, new_dept_id)` with input validation and error handling, to implement **triggers** for salary validation and audit logging, and to test normal and edge cases.

2 Software / Tools Required

Component	Specification
DBMS	MySQL Server 8.0 (Community Edition)
Query Editor	Online MySQL compiler / MySQL Workbench 8.0
Operating System	Windows 11
Language	SQL (DDL, DML, DQL)
Features used	Stored procedures, SIGNAL/RESIGNAL, handlers, triggers

3 Theory

A **stored procedure** is a named block of SQL statements stored in the database and executed with `CALL`. Stored procedures keep business logic in one place, reduce network round-trips and can be granted separately from the underlying tables.

Error handling in MySQL:

Construct	Purpose
`SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = …`	Raise a user-defined error (shown as ERROR 1644)
`DECLARE EXIT HANDLER FOR SQLEXCEPTION`	Catch any error, clean up, then leave the block
`RESIGNAL`	Re-raise the caught error to the caller
`START TRANSACTION … COMMIT / ROLLBACK`	Make the change all-or-nothing

A **trigger** is code that the DBMS runs automatically when an `INSERT`, `UPDATE` or `DELETE` occurs on a table. **BEFORE** triggers can validate or modify `NEW` values and can abort the statement with SIGNAL. **AFTER** triggers see the committed row change, which makes them suited to audit logging. Inside a trigger, `OLD` holds the row before the change and `NEW` holds the row after it.

4 Database / Table Structure

Uses `employee` and `department` from Experiment 3, plus a new audit table:

Table: employee_audit

Column	Data Type	Constraint	Description
`audit_id`	INT	PK, AUTO_INCREMENT	Log entry number
`emp_id`	INT	NOT NULL	Employee affected
`old_dept_id`	INT	—	Department before change
`new_dept_id`	INT	—	Department after change
`action_time`	DATETIME	DEFAULT CURRENT_TIMESTAMP	When it happened
`action`	VARCHAR(50)	—	Description of the action

Object	Type	Timing / Event	Purpose
`transfer_employee`	Procedure	CALL	Validated department transfer
`trg_salary_check_insert`	Trigger	BEFORE INSERT	Reject salary < ₹15,000
`trg_salary_check_update`	Trigger	BEFORE UPDATE	Reject salary < ₹15,000
`trg_audit_dept_change`	Trigger	AFTER UPDATE	Log department changes

5 SQL Commands

```
-- Audit table
CREATE TABLE employee_audit (
    audit_id      INT AUTO_INCREMENT PRIMARY KEY,
    emp_id        INT NOT NULL,
    old_dept_id  INT,
    new_dept_id  INT,
    action_time   DATETIME DEFAULT CURRENT_TIMESTAMP,
    action        VARCHAR(50)
);

DELIMITER $$

-- ------------------------------------------------------------------
-- Stored procedure: transfer_employee(emp_id, new_dept_id)
-- ------------------------------------------------------------------
CREATE PROCEDURE transfer_employee(IN p_emp_id INT, IN p_new_dept_id INT)
BEGIN
    DECLARE v_old_dept INT;
```

```
    -- Any SQL error: undo the work and pass the error to the caller
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    -- Validation 1: employee must exist
    IF NOT EXISTS (SELECT 1 FROM employee WHERE emp_id = p_emp_id) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Employee does not exist';
    END IF;

    -- Validation 2: target department must exist
    IF NOT EXISTS (SELECT 1 FROM department WHERE dept_id = p_new_dept_id) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Department does not exist';
    END IF;

    -- Validation 3: must be a real change
    SELECT dept_id INTO v_old_dept FROM employee WHERE emp_id = p_emp_id;
    IF v_old_dept = p_new_dept_id THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Employee already belongs to this department';
    END IF;

    START TRANSACTION;
        UPDATE employee SET dept_id = p_new_dept_id WHERE emp_id = p_emp_id;
    COMMIT;
END$$

-- -------------------------------------------------------------------
-- Trigger 1 & 2: salary validation on INSERT and UPDATE
-- -------------------------------------------------------------------
CREATE TRIGGER trg_salary_check_insert
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 15000 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Salary cannot be less than 15000';
    END IF;
END$$

CREATE TRIGGER trg_salary_check_update
BEFORE UPDATE ON employee
FOR EACH ROW
BEGIN
     IF NEW.salary < 15000 THEN
         SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Salary cannot be less than 15000';
    END IF;
END$$

-- -------------------------------------------------------------------
-- Trigger 3: audit log whenever an employee changes department
-- -------------------------------------------------------------------
CREATE TRIGGER trg_audit_dept_change
AFTER UPDATE ON employee
```

```
FOR EACH ROW
BEGIN
    IF NOT (OLD.dept_id <=> NEW.dept_id) THEN
        INSERT INTO employee_audit (emp_id, old_dept_id, new_dept_id, action)
        VALUES (OLD.emp_id, OLD.dept_id, NEW.dept_id, 'Employee Transferred');
    END IF;
END$$

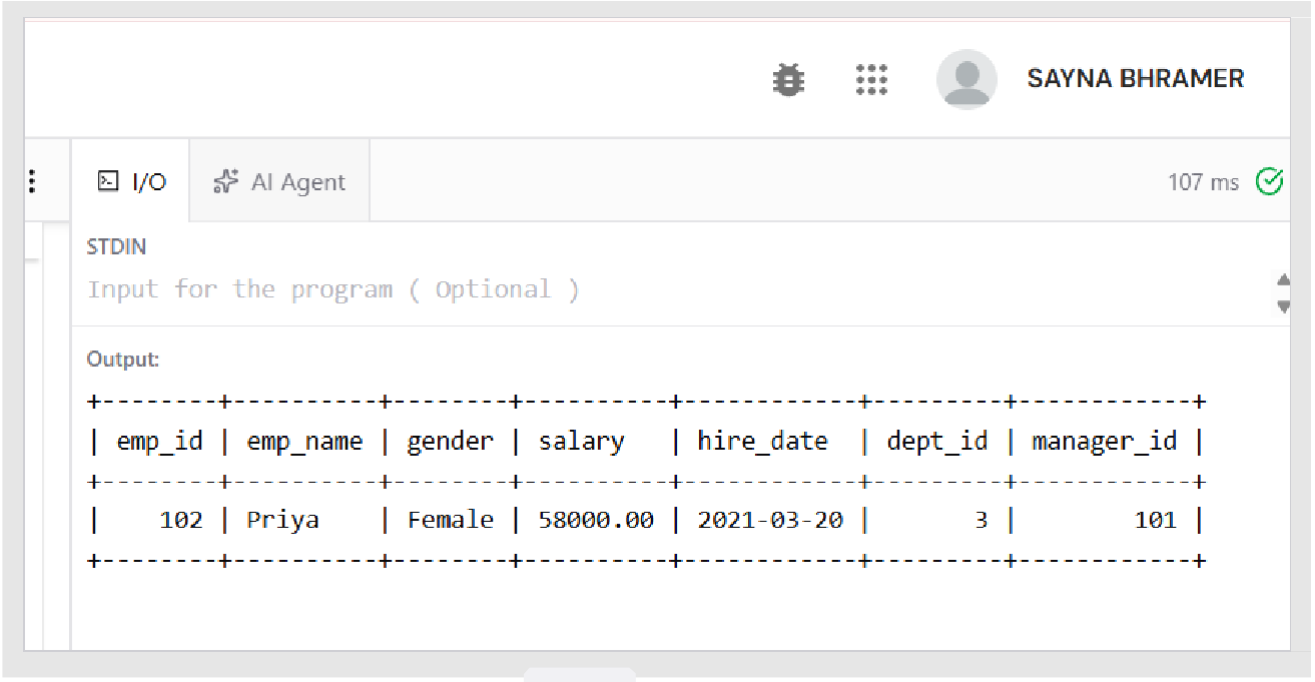
DELIMITER ;
```

6 Output

Q6.1 Valid transfer — move Priya (102) from IT (1) to Finance (3)

```
CALL transfer_employee(102, 3);

SELECT * FROM employee WHERE emp_id = 102;
```



Q6.2 Audit trigger fired automatically

```
SELECT * FROM employee_audit;
```



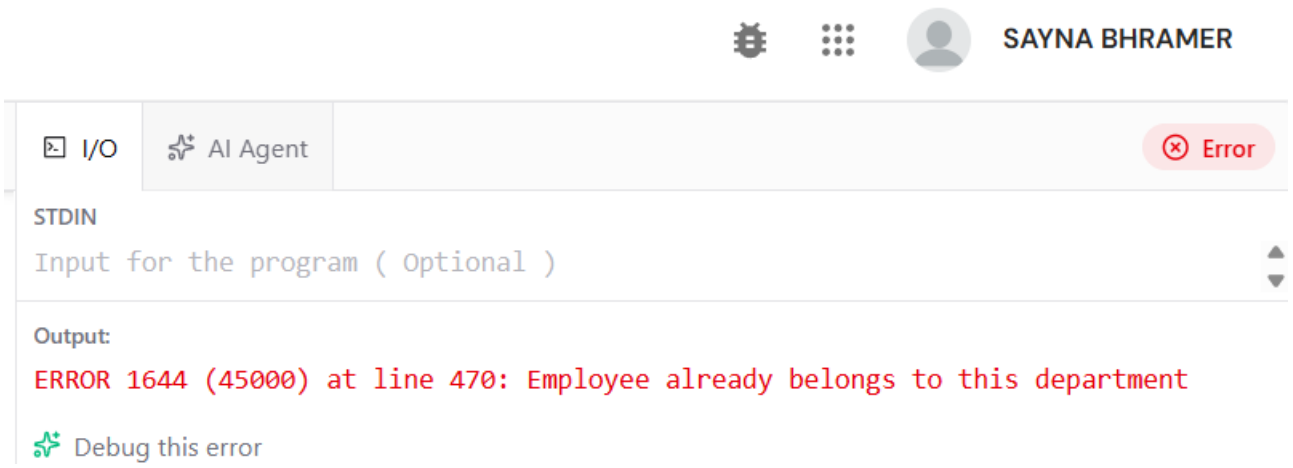
Q6.3 Salary trigger — update below the minimum is blocked

```
UPDATE employee SET salary = 12000 WHERE emp_id = 105;
```



Q6.4 Edge case — employee does not exist

```
CALL transfer_employee(999, 2);
-- ERROR 1644 (45000): Employee does not exist
```



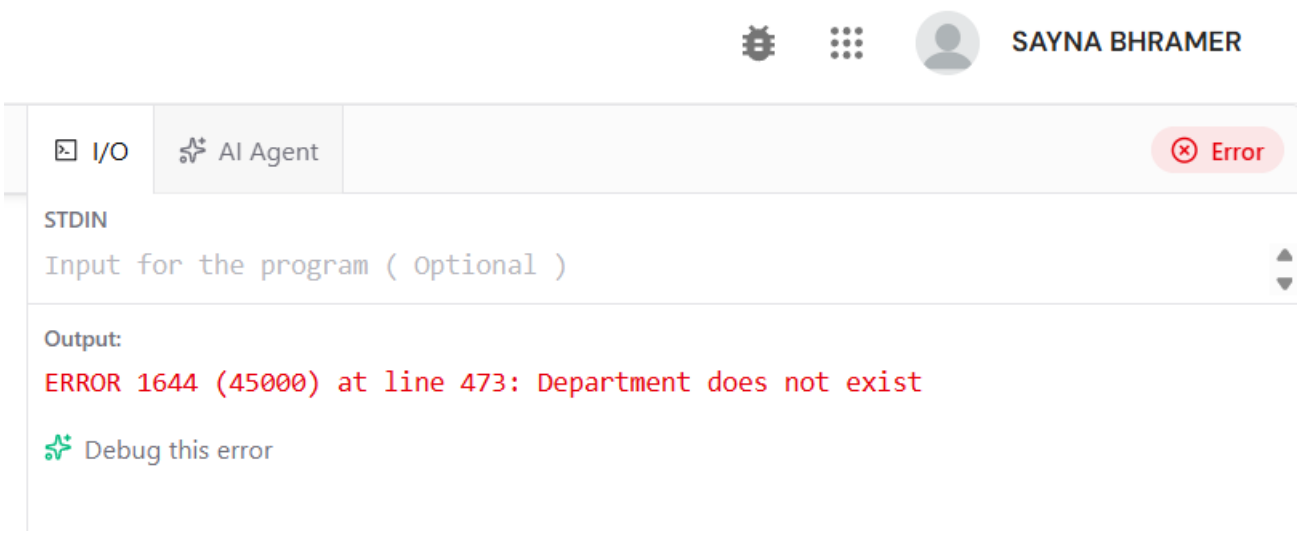
Q6.5 Edge case — department does not exist

```
CALL transfer_employee(101, 9);
-- ERROR 1644 (45000): Department does not exist
```



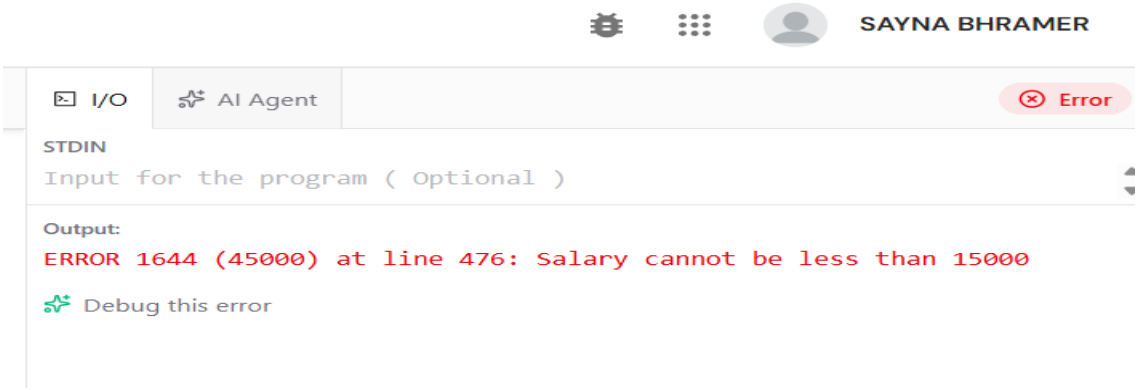
Q6.6 Edge case — transfer to the same department

```
CALL transfer_employee(103, 1);
-- ERROR 1644 (45000): Employee already belongs to this department
```



Q6.7 Edge case — INSERT with invalid salary

```
INSERT INTO employee VALUES
    (131, 'Test', 'Male', 10000.00, '2026-09-27', 1, 101);
-- ERROR 1644 (45000): Salary cannot be less than 15000
```



Test case	Input	Expected	Status
Valid transfer	(102, 3)	Row updated + audit entry	✓ Pass
Salary below minimum	salary = 12000	ERROR 1644: Salary cannot be less than 15000	✓ Pass
Unknown employee	(999, 2)	ERROR 1644: Employee does not exist	✓ Pass
Unknown department	(101, 9)	ERROR 1644: Department does not exist	✓ Pass
Same department	(103, 1)	ERROR 1644: already belongs to this department	✓ Pass
Invalid INSERT salary	10000	ERROR 1644: Salary cannot be less than 15000	✓ Pass

7 Result / Conclusion

The stored procedure `transfer employee` moved employee 102 (Priya) from IT to Finance, and the AFTER UPDATE trigger automatically wrote the audit record (1 → 3, "Employee Transferred", with a timestamp). The BEFORE triggers rejected salaries below ₹15,000 with a custom SQLSTATE 45000 error, leaving the data unchanged. The edge-case tests (unknown employee, unknown department, same-department transfer, invalid insert) were each caught by explicit validation.